

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «КиСБ»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4 курса
Группы АС-65
Ракецкий П. П.
Проверил:
Хацкевич А. С.

Цель: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 3

1. Реализовать приложение, генерирующее псевдослучайную равномерно распределенную последовательность произвольной длины, подобрать параметры генератора с максимальным значением периода последовательности, проверить сгенерированную последовательность на равномерное распределение.

Генератор: Эйхенауэра-Лена с обращением, размер алфавита: $N=20$.

2. Реализовать приложение, позволяющее генерировать простое число по следующей схеме: Сгенерировать случайное p -битовое число n , установить старший и младший биты равные 1, n не должно делиться на небольшие простые числа по типу 3,5,7,11 и т.д., выполнить один тест Рабина-Миллера проверки простоты числа для некоторого случайного a .

Алгоритм: Полиномиальный тест распознавания простоты.

Код программы:

```
import math
import random
import secrets
import time
from collections import Counter

class EichenauerLenaGenerator:
    def __init__(self, modulus=20, a=7, c=3, seed=3):
        self.modulus = modulus
        self.a = a % modulus
        self.c = c % modulus
        self.state = seed % modulus

    def _inverse(self, value):
        try:
            return pow(value, -1, self.modulus)
        except ValueError:
            return 0

    def next_int(self):
        self.state = (self.a * self._inverse(self.state) + self.c) % self.modulus
        return self.state

    def sequence(self, length):
        return [self.next_int() for _ in range(length)]

def sequence_period(generator, limit=10000):
    seen = {}
    state = generator.state
    for i in range(limit):
        if state in seen:
            return seen[state], i - seen[state]
        seen[state] = i
        state = (generator.a * generator._inverse(state) + generator.c) % generator.modulus
```

```

        return None, None

def perfect_power(n):
    if n < 4:
        return False
    max_exp = n.bit_length()
    for exponent in range(2, max_exp + 1):
        low, high = 2, 1 << ((n.bit_length() + exponent - 1) // exponent)
        while low <= high:
            mid = (low + high) // 2
            value = mid ** exponent
            if value == n:
                return True
            if value < n:
                low = mid + 1
            else:
                high = mid - 1
    return False

def gcd(a, b):
    while b:
        a, b = b, a % b
    return abs(a)

def multiplicative_order_exceeds(n, r):
    if gcd(n, r) != 1:
        return False
    value = 1
    for k in range(1, r * r + 1):
        value = (value * n) % r
        if value == 1:
            return k > int(math.log2(n) ** 2)
    return True

def find_aks_r(n):
    limit = max(2, int(math.log2(n) ** 2))
    for r in range(2, n):
        if gcd(n, r) == 1 and multiplicative_order_exceeds(n, r):
            return r
        if r > limit * 10 and r > n.bit_length() ** 3:
            break
    return None

def poly_multiply_mod(a, b, modulus, r):
    result = [0] * (2 * r - 1)
    for i, x in enumerate(a):
        if x:
            for j, y in enumerate(b):
                if y:
                    result[i + j] = (result[i + j] + x * y) % modulus
    for degree in range(2 * r - 2, r - 1, -1):
        coefficient = result[degree] % modulus
        if coefficient:
            result[degree] = 0
            result[degree - r] = (result[degree - r] + coefficient) % modulus
    return result[:r]

def poly_power_mod(base, exponent, modulus, r):
    result = [1] + [0] * (r - 1)
    while exponent:
        if exponent & 1:
            result = poly_multiply_mod(result, base, modulus, r)
        exponent >>= 1
    if exponent:

```

```

        base = poly_multiply_mod(base, base, modulus, r)
    return result

def aks_is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n == p:
            return True
        if n % p == 0:
            return False
    if perfect_power(n):
        return False

    r = find_aks_r(n)
    if r is None:
        return trial_division_is_prime(n)
    for a in range(2, min(r, n)):
        common = gcd(a, n)
        if 1 < common < n:
            return False
    if n <= r:
        return True
    limit = int(math.sqrt(r) * math.log2(n))
    for a in range(1, limit + 1):
        base = [a % n, 1] + [0] * (r - 2)
        left = poly_power_mod(base, n, n, r)
        right = [0] * r
        right[0] = a % n
        right[n % r] = (right[n % r] + 1) % n
        if left != right:
            return False
    return True

def trial_division_is_prime(n):
    if n < 2:
        return False
    if n % 2 == 0:
        return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return False
        d += 2
    return True

def sieve_primes(limit=2000):
    sieve = bytearray(b'\x01') * (limit + 1)
    sieve[0:2] = b'\x00\x00'
    for p in range(2, int(limit ** 0.5) + 1):
        if sieve[p]:
            start = p * p
            sieve[start:limit + 1:p] = b'\x00' * ((limit - start) // p + 1)
    return [i for i, flag in enumerate(sieve) if flag]

def random_candidate(bits, generator):
    mask = 0
    for _ in range(bits):
        mask = (mask << 1) | (generator.next_int() & 1)
    value = secrets.randbits(bits) ^ mask
    value |= (1 << (bits - 1))
    value |= 1
    return value

```

```

def generate_prime(bits=16, max_candidates=100000):
    generator = EichenauerLenaGenerator()
    small_primes = sieve_primes(2000)
    stats = {'checked': 0, 'filtered': 0, 'aks_checked': 0}
    started = time.perf_counter()
    for _ in range(max_candidates):
        candidate = random_candidate(bits, generator)
        stats['checked'] += 1
        rejected = False
        for p in small_primes:
            if candidate == p:
                stats['aks_checked'] += 1
                stats['elapsed'] = time.perf_counter() - started
                return candidate, stats
            if candidate % p == 0:
                stats['filtered'] += 1
                rejected = True
                break
        if rejected:
            continue
        stats['aks_checked'] += 1
        if aks_is_prime(candidate):
            stats['elapsed'] = time.perf_counter() - started
            return candidate, stats
    stats['elapsed'] = time.perf_counter() - started
    return None, stats

def show_generator_demo():
    generator = EichenauerLenaGenerator()
    seq = generator.sequence(1000)
    counts = Counter(seq)
    start, period = sequence_period(EichenauerLenaGenerator())
    print('\n=== Этап 1. Генератор Эйхенауэра – Лена с обращением ===')
    print(f'Параметры: N={generator.modulus}, a={generator.a}, c={generator.c}, x0=3')
    print('Первые 30 значений:', seq[:30])
    print(f'Уникальных значений в выборке: {len(counts)} из {generator.modulus}')
    print(f'Частоты значений 0..{generator.modulus - 1}:')
    for value in range(generator.modulus):
        print(f'    {value:2}: {counts.get(value, 0):4} ({counts.get(value, 0) / len(seq):6.2%})')
    print(f'Цикл при текущих параметрах: начало={start}, длина={period}')
    print('Примечание: при N=20 обратный элемент существует не для всех остатков;')
    print('для необратимых элементов в этой реализации принято inv(x)=0.')

def main():
    print('Лабораторная работа №1 – вариант 3')
    while True:
        print('\n1 – генерация и статистика последовательности')
        print('2 – проверить число на простоту (AKS)')
        print('3 – сгенерировать простое p-битное число')
        print('0 – выход')
        choice = input('Выберите действие: ').strip()
        if choice == '1':
            show_generator_demo()
        elif choice == '2':
            try:
                n = int(input('Введите целое число n: '))
                started = time.perf_counter()
                result = aks_is_prime(n)
                print(f'{n} – ' + ('простое' if result else 'составное'))
                print(f'Время проверки: {time.perf_counter() - started:.6f}')
            except ValueError:
                pass

```

```

c')
        except ValueError:
            print('Ошибка: необходимо ввести целое число.')
    elif choice == '3':
        try:
            bits = int(input('Разрядность числа в битах (например, 16 или
20): '))
            if bits < 2:
                print('Разрядность должна быть не меньше 2.')
                continue
            prime, stats = generate_prime(bits)
            if prime is None:
                print('Простое число не найдено за отведённое число
попыток.')
            else:
                print(f'Найдено простое число: {prime}
({prime.bit_length()} бит)')
                print(f"Кандидатов проверено: {stats['checked']}")
                print(f"Отсеяно малым делением: {stats['filtered']}")
                print(f"Кандидатов передано в AKS: {stats['aks_checked']}")
                print(f"Время: {stats['elapsed']:.6f} c")
            except ValueError:
                print('Ошибка: введите целую разрядность.')
        elif choice == '0':
            break
        else:
            print('Неизвестная команда.')
if __name__ == '__main__':
    main()

```

Вывод программы в консоле:

```

=== Этап 1. Генератор Эйхенауэра – Лена с обращением ===
Параметры: N=20, a=7, c=3, x0=3
Первые 30 значений: [12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3, 12, 3]
Уникальных значений в выборке: 2 из 20
Частоты значений 0..19:
  0:   0 ( 0.00%)
  1:   0 ( 0.00%)
  2:   0 ( 0.00%)
  3: 500 (50.00%)
  4:   0 ( 0.00%)
  5:   0 ( 0.00%)
  6:   0 ( 0.00%)
  7:   0 ( 0.00%)
  8:   0 ( 0.00%)
  9:   0 ( 0.00%)
 10:   0 ( 0.00%)
 11:   0 ( 0.00%)
 12: 500 (50.00%)
 13:   0 ( 0.00%)
 14:   0 ( 0.00%)
 15:   0 ( 0.00%)
 16:   0 ( 0.00%)
 17:   0 ( 0.00%)
 18:   0 ( 0.00%)
 19:   0 ( 0.00%)

Цикл при текущих параметрах: начало=0, длина=2
Примечание: при N=20 обратный элемент существует не для всех остатков;
для необратимых элементов в этой реализации принято inv(x)=0.

```

Введите целое число n: 15
15 – составное
Время проверки: 0.000064 с

Разрядность числа в битах (например, 16 или 20): 16
Найдено простое число: 42677 (16 бит)
Кандидатов проверено: 11
Отсеяно малым делением: 10
Кандидатов передано в AKS: 1
Время: 57.175288 с

Вывод: освоил программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.